DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de Software

Análisis y Diseño de Software - NRC 30724

Guía 1: PlantUML, CU, Análisis

Proyecto:

Sistema de Gestión y Control de Transporte de Carga Pesada

Grupo

Saraguro Leidy

Ñacato Alexander

Jaguaco Jonathan

Obando Erick

Instructor: Ing. Jenny Alexandra Ruiz Robalino

Fecha: 05/05/2026

TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

1. Objetivo

Modelar el diagrama de casos de uso del análisis del Sistema de Gestión y Control de Transporte de Carga Pesada en PlantUML, partiendo de los 4 requisitos funcionales de nivel 0 y aplicando las relaciones <<include>> justificadas, con el fin de establecer la matriz de trazabilidad entre requisitos, actores, casos de uso y criterios de aceptación con el fin de usarlo como base para el posterior análisis estructural y dinámico del sistema.

2. Desarrollo

2.1. Requisitos Funcionales

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema valida las credenciales de un usuario (nombre de usuario y contraseña) para permitirle el acceso a las funcionalidades protegidas, según su rol asignado.	Usuario	Iniciar Sesión
RF-04	El sistema permite al coordinador administrar transportistas, al transportista registrar su documentación y a la secretaría consultar información y gestionar notificaciones, asegurando el control y la validez de los datos.	Coordinador Transportista Secretaria	Gestionar Transportistas
RF-05	El sistema permite a la secretaría y al gerente crear, consultar, reprogramar y cancelar viajes. Además, procesa la información, define rutas, asigna transportistas y mantiene actualizado el estado del viaje para su control y seguimiento.	Secretaria Gerente Transportista Cliente	Gestionar Viajes
RF-06	El sistema permite al coordinador monitorear viajes en ejecución, visualizar información de ruta y estado del viaje, y gestionar alertas o notificaciones al transportista ante incidencias o retrasos detectados.	Coordinador Transportista	Monitorear Viajes

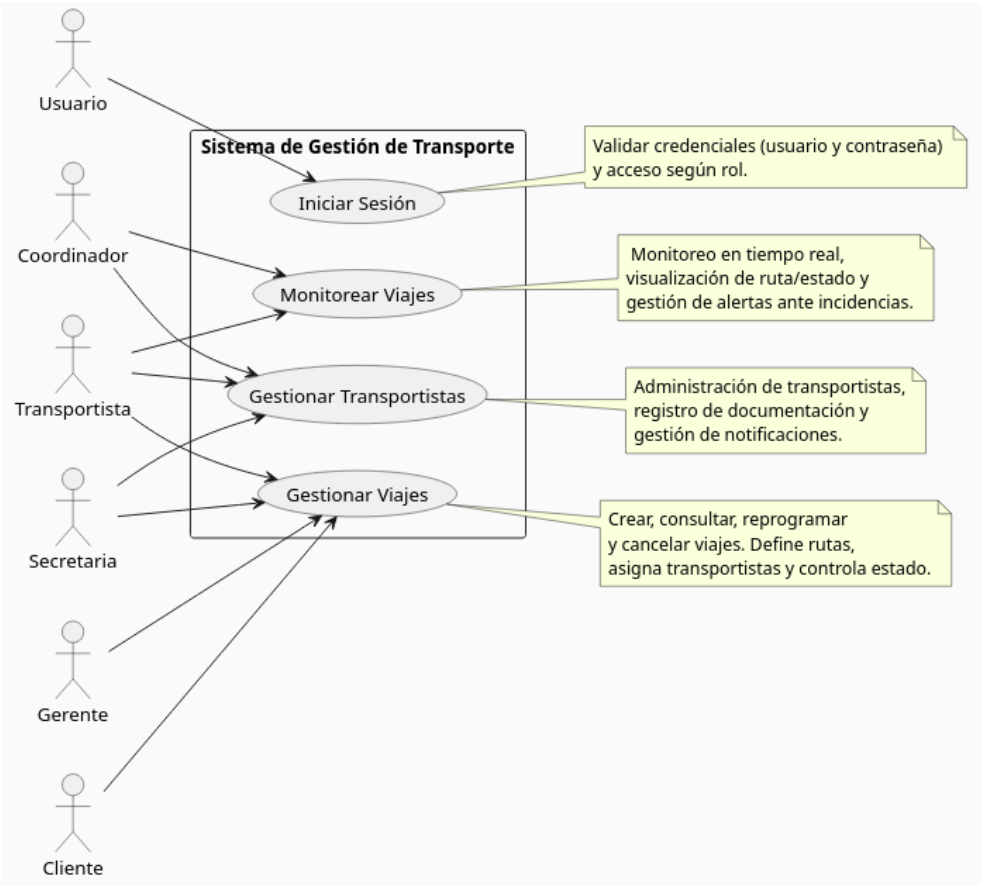
TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

2.2. Diagrama de Casos de Uso

```
@startuml
left to right direction

actor Usuario as U
actor Coordinador as C
actor Secretaria as S
actor Gerente as G
actor Transportista as T
actor Cliente as CL

rectangle "Sistema de Gestión de Transporte" {
  (Iniciar Sesión) as UC1
  (Gestionar Transportistas) as UC2
  (Gestionar Viajes) as UC3
  (Monitorear Viajes) as UC4
}

note right of UC1
  Validar credenciales (usuario y contraseña)
  y acceso según rol.
end note

note right of UC2
  Administración de transportistas,
  registro de documentación y
  gestión de notificaciones.
end note

note right of UC3
  Crear, consultar, reprogramar
  y cancelar viajes. Define rutas,
  asigna transportistas y controla estado.
end note

note right of UC4
   Monitoreo en tiempo real,
  visualización de ruta/estado y
  gestión de alertas ante incidencias.
end note

U --> UC1
C --> UC2
```

TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
S --> UC2
T --> UC2
S --> UC3
G --> UC3
T --> UC3
CL --> UC3
C --> UC4
T --> UC4

@enduml
```



Explicación Breve: El diagrama de casos de uso de nivel 0 muestra las principales funcionalidades del sistema y los actores que interactúan con ellas. Cada caso de uso representa un requisito funcional clave, como iniciar sesión, gestionar transportistas, viajes y monitoreo. Los actores se distribuyen a ambos lados para mejorar la claridad de sus interacciones.

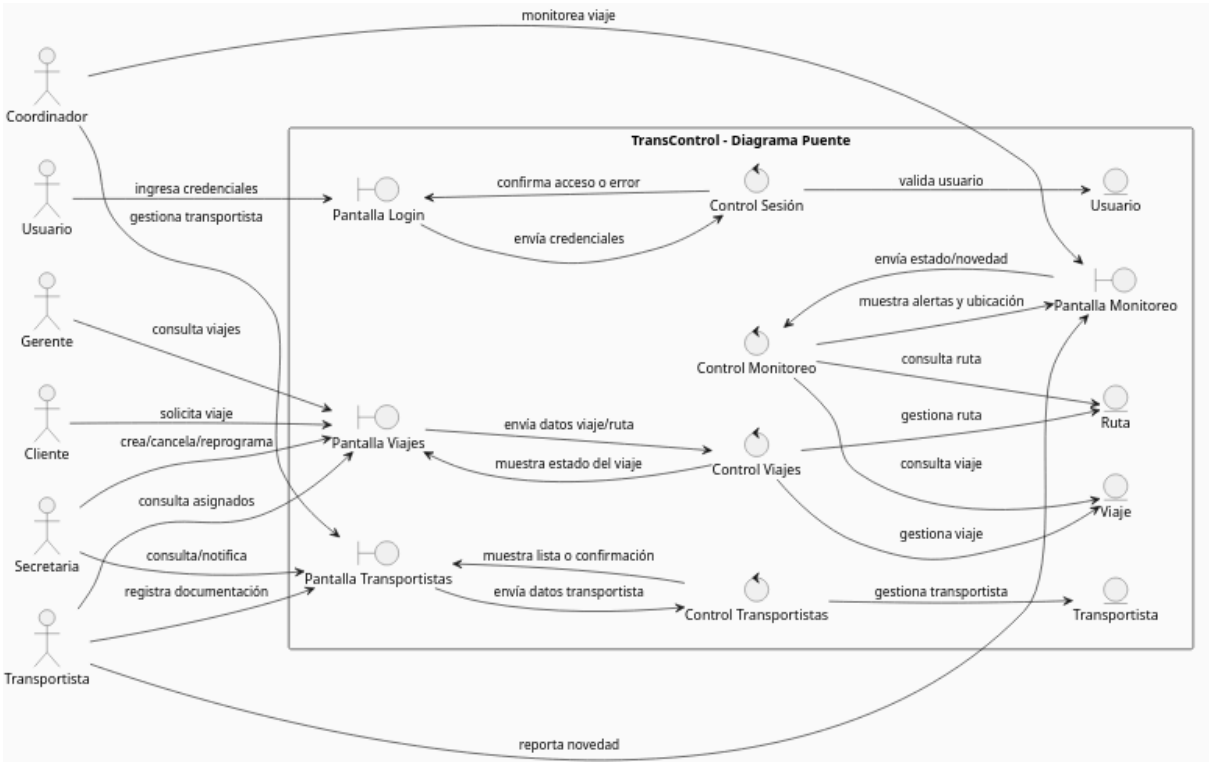
TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

2.3. Puente

```
@startuml
left to right direction

actor Usuario as U
actor Coordinador as C
actor Secretaria as S
actor Gerente as G
actor Transportista as T
actor Cliente as CL

rectangle "TransControl - Diagrama Puente" {

  ' UC1 - Iniciar Sesión
  boundary "Pantalla Login" as B1
  control "Control Sesión" as CT1
  entity "Usuario" as E1

  ' UC2 - Gestionar Transportistas
  boundary "Pantalla Transportistas" as B2
  control "Control Transportistas" as CT2
  entity "Transportista" as E2

  ' UC3 - Gestionar Viajes
  boundary "Pantalla Viajes" as B3
  control "Control Viajes" as CT3
  entity "Viaje" as E3
  entity "Ruta" as E4

  ' UC4 - Monitorear Viajes
  boundary "Pantalla Monitoreo" as B4
  control "Control Monitoreo" as CT4

  ' Flujos de entrada
  B1 --> CT1 : envía credenciales
  B2 --> CT2 : envía datos transportista
  B3 --> CT3 : envía datos viaje/ruta
  B4 --> CT4 : envía estado/novedad

  ' Flujos hacia entidades
  CT1 --> E1 : valida usuario
  CT2 --> E2 : gestiona transportista
  CT3 --> E3 : gestiona viaje
  CT3 --> E4 : gestiona ruta
  CT4 --> E3 : consulta viaje
```

TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

```
  CT4 --> E4 : consulta ruta

  ' Flujos de retorno
  CT1 --> B1 : confirma acceso o error
  CT2 --> B2 : muestra lista o confirmación
  CT3 --> B3 : muestra estado del viaje
  CT4 --> B4 : muestra alertas y ubicación
}

' Actores a boundaries
U --> B1 : ingresa credenciales
C --> B2 : gestiona transportista
S --> B2 : consulta/notifica
T --> B2 : registra documentación
S --> B3 : crea/cancela/reprograma
G --> B3 : consulta viajes
T --> B3 : consulta asignados
CL --> B3 : solicita viaje
C --> B4 : monitorea viaje
T --> B4 : reporta novedad

@enduml
```



TALLER 2 **UNIDAD 1**	**Departamento:** Ciencias de la computación
	Carrera: Ingeniería de software **Asignatura:** Análisis y Diseño de software **NRC:** 30724
	Apellidos y nombres del estudiante: Saraguro Leidy, Ñacato Alexander, Jaguaco Jonathan, Obando Erick

Explicación Breve: El diagrama puente conecta el diagrama de casos de uso con el análisis, mostrando cómo los actores (Usuario, Coordinador, Secretaria, Gerente, Transportista y Cliente) interactúan con las pantallas boundary para ejecutar las operaciones de gestión de sesión, transportistas, viajes y monitoreo; cada pantalla envía los datos al control correspondiente quien los gestiona con las entidades (Usuario, Transportista, Viaje y Ruta) y retorna una confirmación o resultado, sirviendo así como base para construir el diagrama de clases de análisis.

2.4. Explicación breve de casos de uso

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
Iniciar Sesión	Usuario	El usuario debe estar registrado con rol asignado	Ingresar credenciales → validar → otorgar acceso según rol	Acceso concedido o error mostrado
Gestionar Transportistas	Coordinador / Secretaria / Transportista	El usuario debe haber iniciado sesión	Seleccionar operación → ingresar datos → validar documentación → guardar o notificar	Transportista, credenciales y viajes administrados correctamente
Gestionar Viajes	Secretaria / Gerente / Transportista / Cliente	El usuario debe haber iniciado sesión y existir al menos un viaje en ejecución con un transportista asignado.	Seleccionar operación → ingresar datos del viaje → definir ruta → asignar transportista → guardar	Viaje creado, consultado, reprogramado o cancelado
Monitorear Viajes	Coordinador / Transportista	Debe existir al menos un viaje en ejecución	Seleccionar viaje → visualizar GPS → detectar incidencias → gestionar alertas	Viaje monitoreado y alertas gestionadas

2.5. Matriz de Trazabilidad

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Iniciar Sesión	Usuario	Alta	El sistema valida credenciales y otorga acceso según el rol asignado; credenciales incorrectas muestran mensaje de error.	UC1
RF-04	Gestionar Transportistas	Coordinador / Secretaria / Transportista	Alta	El sistema registra, edita o elimina un transportista con documentación válida y notifica inconsistencias.	UC2
RF-05	Gestionar Viajes	Secretaria / Gerente / Transportista / Cliente	Alta	El sistema crea, consulta, reprograma o cancela un viaje con ruta y transportista asignado correctamente.	UC3
RF-06	Monitorear Viajes	Coordinador / Transportista	Media	El sistema muestra la ubicación GPS en tiempo real y genera alertas ante retrasos o desvíos detectados.	UC4

3. Conclusiones

- La implementación del diagrama puente para TransControl nos permitió comprender cómo cada caso de uso de nivel 0 (Iniciar Sesión, Gestionar Transportistas, Gestionar Viajes y Monitorear Viajes) se descompone en tres elementos BCE: un Boundary que representa la pantalla de interacción, un Control que procesa la lógica y una Entity que

persiste los datos, estableciendo así la base estructural para el diagrama de clases de análisis.

- La matriz de trazabilidad construida evidenció que cada RF debe vincularse directamente con su caso de uso, actor, prioridad y criterio de aceptación, lo que garantiza que ningún requisito quede sin representación en el modelo y que cada criterio de aceptación pueda convertirse directamente en un caso de prueba verificable.
- En conclusión, el diagrama puente actúa como eslabón central entre el diagrama de casos de uso y el diagrama de clases, ya que los Boundary se convierten en clases de vista, los Control en clases de servicio y las Entity en clases de modelo, comprendiendo que cualquier modificación en los casos de uso obliga a revisar el puente y posteriormente el diagrama de clases para mantener la trazabilidad del sistema.

4. Recomendaciones:

- Para futuros talleres se recomienda definir primero la tabla de casos de uso con su precondición, flujo y resultado esperado antes de escribir el código PlantUML, ya que esto evita agregar actores o relaciones que no tienen respaldo en los requisitos funcionales del SRS y reduce correcciones posteriores en el diagrama.
- Se sugiere verificar que el flujo BCE sea siempre bidireccional desde el inicio, es decir que por cada flecha de entrada del actor hacia el Boundary exista una flecha de retorno del Control hacia el Boundary con la confirmación o resultado, pues omitir el retorno deja el ciclo de interacción incompleto y no refleja el comportamiento real del sistema.
- También es útil mantener el SRS visible mientras se construye el diagrama puente, verificando que cada Boundary, Control y Entity tenga correspondencia exacta con un RF documentado, para que la evidencia PlantUML de la matriz de trazabilidad sea consistente y defendible ante la rúbrica de evaluación.